

THE **AI** CODEBASE ANALYST

FROM ZERO TO A WORKING AGENT

LESSON 8
HANDLING FAILURE GRACEFULLY

Lesson 8: Handling Failure Gracefully

"Antifragility is beyond resilience or robustness. The resilient resists shocks and stays the same; the antifragile gets better."

— Nassim Nicholas Taleb

The "Why": The Fragile Lab Specimen

We have reached a pivotal moment. Our AI agent is intelligent, observable, reliable in its output, and secure. It stands proudly on our Three Pillars. Yet, it has a hidden, fatal flaw: **it is fragile**.

Our agent has been developed in a perfect "lab" environment where files always exist, networks never drop, and APIs always respond instantly. This is not the real world. In a production environment, **entropy reigns**. What happens if...

- ...a user tries to query the agent before the `create_vectorstore.py` script has been run?
- ...the OpenRouter API has a momentary network glitch during a call?
- ...a tool call is made repeatedly, costing money and time for the same result?

Right now, in all these cases, our agent will ungracefully crash or perform inefficiently. A prototype can be fragile; a production system must be **resilient**. It must anticipate failure and handle it with grace. How do we transform our fragile lab specimen into an antifragile system that can withstand real-world chaos?

The "What": Architecting for Resilience

We will fortify our agent by weaving resilience directly into its components. This is not about fixing bugs; it's about architecting for failure as a core design principle. We will implement three powerful, professional patterns:

Pattern	Problem It Solves	Implementation
Graceful Failure & Lazy Loading	The app crashes if a required file (like the vectorstore) is missing on startup.	We'll move the file-loading logic <i>inside</i> the tool's function call and wrap it in a try...except block.
Automatic Retries	Transient problems like a network flicker cause the entire operation to fail.	We'll add a <code>@retry</code> decorator (using the tenacity library) that automatically re-attempts a failing network call a few times.
Defensive Caching	Repeatedly calling an expensive, deterministic tool (like our analyzer) wastes time and money.	We'll add a <code>@lru_cache</code> decorator that stores the results of a function call and returns the cached result instantly on subsequent identical calls.

The "How" (The Build): Step-by-Step

Let's refactor our existing tools to make them robust.

1. Update Your Dependencies

The automatic retry pattern is best implemented with a dedicated library. We will add tenacity to our project.

Update your requirements.txt file:

```
... (previous libraries) ...  
unstructured  
tenacity # <-- NEW: For robust retries
```

Then, run the installer from your activated (venv) terminal:

```
pip install -r requirements.txt
```

2. Refactor the Retriever Tool (Lazy Loading & Graceful Failure)

We'll fix the critical anti-pattern of performing file I/O at import time. The vectorstore will now be loaded "lazily"—only when the tool is actually called.

- **File:** tools/retriever_tool.py (MODIFIED)

```
# tools/retriever_tool.py  
  
import os  
from pathlib import Path  
from langchain.tools import tool  
from langchain_community.vectorstores import FAISS  
from langchain_huggingface import HuggingFaceEmbeddings  
from pydantic import BaseModel, Field  
  
# --- 1. Security Sandbox Configuration ---  
# This block defines the "jail" or "sandbox" for our tool. It calculates  
# the absolute path to the project's 'source_code' directory. Any  
# attempt  
# by the tool to access files outside this directory will be blocked.  
PROJECT_ROOT = Path(__file__).resolve().parent.parent  
ALLOWED_DIRECTORY = PROJECT_ROOT / "source_code"  
VECTORSTORE_PATH = PROJECT_ROOT / "vectorstore"  
  
# --- 2. Tool Input Schema ---  
# Defines the expected input for our tool using Pydantic. This provides
```

```

# clear validation, typing, and documentation for the tool's interface.
class RetrieverInput(BaseModel):
    query: str = Field(description="The query to search for in the
codebase.")

# --- 3. Tool Implementation: The Resilient Retriever ---
@tool("codebase-retriever", args_schema=RetrieverInput)
def codebase_retriever(query: str) -> str:
    """
    Searches the codebase to find relevant code snippets based on the
query.
    This tool is hardened with three professional patterns:
    1. Lazy Loading: Loads the vectorstore only when needed.
    2. Sandboxing: Validates file paths to prevent unauthorized access.
    3. Graceful Failure: Handles errors without crashing the agent.
    """
    try:
        # --- 3A. Lazy Loading the Vectorstore ---
        # The expensive I/O operation to load the vectorstore happens
here, inside
        # the tool call. This ensures the main application starts
instantly.
        print("\n--- Lazily loading vectorstore... ---")
        embeddings = HuggingFaceEmbeddings(
            model_name="sentence-transformers/all-MiniLM-L6-v2"
        )
        db = FAISS.load_local(
            str(VECTORSTORE_PATH), embeddings,
allow_dangerous_deserialization=True
        )
        retriever = db.as_retriever()
        print("--- Vectorstore loaded successfully. ---")

        retrieved_docs = retriever.invoke(query)

        # --- 3B. Process and Securely Validate Each Document ---
        formatted_docs = []
        for i, doc in enumerate(retrieved_docs):
            source_path_str = doc.metadata.get("source", "Unknown")

```

```

# The SANDBOX VALIDATION logic from Lesson 7 remains.
try:
    resolved_path = Path(source_path_str).resolve()
    if not resolved_path.is_relative_to(ALLOWED_DIRECTORY):
        print(f"⚠ SECURITY WARNING: Attempted to access
restricted path: {source_path_str}")
        formatted_docs.append(f"--- ACCESS DENIED: Cannot
display content from restricted path: {source_path_str} ---")
        continue
    except Exception:
        print(f"⚠ SECURITY WARNING: Invalid source path
encountered: {source_path_str}")
        formatted_docs.append(f"--- ACCESS DENIED: Invalid
source path ---")
        continue

    formatted_docs.append(f"--- Code Snippet {i + 1} (Source:
{source_path_str}) ---\n{doc.page_content}")

    if not formatted_docs:
        return "No relevant and accessible code snippets found in
the 'source_code' directory."

    return "\n\n".join(formatted_docs)

except Exception as e:
    # --- 3C. Graceful Failure Handling ---
    # If any part of the process fails (e.g., the vectorstore
    doesn't exist),
    # this block prevents the entire agent from crashing and
    provides a
    # helpful error message. This is a key resiliency pattern.
    print(f"--- Error during vectorstore load or retrieval: {e}
---")
    return "Error: Could not access the codebase vectorstore. It may
not have been created yet. Please run the indexing script."

```

Deconstructing the Script

This script refactors our retriever tool to be resilient against real-world failures. It introduces two critical engineering patterns: Lazy Loading and Graceful Failure.

Lazy Loading (Section 3A)

Previously, our script performed a file I/O operation (`FAISS.load_local`) the moment it was imported. This is an anti-pattern because it makes the application fragile—if the file is missing, the entire program crashes on startup. We've moved this logic *inside* the tool's function call. This is **Lazy Loading**. The benefit is twofold: our application starts faster, and the performance cost of loading the database is only paid when the tool is actually used.

Graceful Failure (Section 3C)

The entire tool's logic is now wrapped in a `try...except` block. This is our **Graceful Failure** mechanism. If an error occurs during the loading process (like the `FileNotFoundError` we simulate in our test), the `except` block catches it. Instead of crashing, the tool returns a helpful, human-readable error message. The agent can then use this information to inform the user what went wrong, making the entire system more robust and user-friendly.

3. Refactor the Analyzer Tool (Retries & Caching)

Next, we'll fortify our expensive, network-dependent analyzer tool. To do this without breaking LangChain's ability to inspect our tool, we'll use a professional design pattern that separates the public interface from the internal, decorated implementation.

- **File:** `tools/structured_analyzer_tool.py` (MODIFIED)

```
# tools/structured_analyzer_tool.py

import os
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field
from schemas.function_analysis import FunctionAnalysis

# --- 1. Import Resilience Decorators ---
# `lru_cache` provides in-memory caching for function calls.
# `tenacity` provides a powerful `@retry` decorator for handling
transient errors.
```

```
from functools import lru_cache
from tenacity import retry, stop_after_attempt, wait_exponential

# --- 2. Tool Input Schema ---
class AnalyzerToolInput(BaseModel):
    code_snippet: str = Field(description="A string containing a single,
complete Python function to be analyzed.")

# --- 3. The Public Interface ---
# This is the clean, public-facing function that LangChain's @tool
decorator
# can easily inspect to understand its signature and purpose. Its only
job
# is to delegate the actual work to our decorated internal function.
@tool("structured-code-analyzer", args_schema=AnalyzerToolInput)
def structured_code_analyzer(code_snippet: str) -> FunctionAnalysis:
    """
    Analyzes a Python function's code snippet and returns a structured
analysis.
    This tool is cached and will retry up to 3 times on network failure.
    Use this when you need a detailed, structured breakdown of a
specific function.
    """
    return _analyze_with_retry_and_cache(code_snippet)

# --- 4. The Decorated Internal Implementation ---
# This "private" function contains the core logic. We apply our
resilience
# and performance decorators here, hidden from the main @tool decorator.

@lru_cache(maxsize=32) # Cache up to 32 recent, unique results.
@retry(
    stop=stop_after_attempt(3), # Retry the function up to 3 times if it
fails.
    wait=wait_exponential(multiplier=1, min=1, max=10) # Wait 1s, then
2s, then 4s... between retries.
)
def _analyze_with_retry_and_cache(code_snippet: str) ->
FunctionAnalysis:
    """Internal function to perform the analysis with caching and
```

```
retries."""
    # This print statement is a simple way to prove that our cache is
    working.
    # It will only appear on a "cache miss" (the first time a unique
    input is seen).
    print("\n--- Calling structured_code_analyzer (Cache miss or new
    input) ---")

    llm = ChatOpenAI(
        base_url="https://openrouter.ai/api/v1",
        api_key=os.getenv("OPENROUTER_API_KEY"),
        model="google/gemini-2.5-flash",
        temperature=0
    )

    structured_llm = llm.with_structured_output(FunctionAnalysis)
    prompt = f"Analyze this Python function and describe its purpose,
    arguments, and return value. Here is the function:
    ```python\n{code_snippet}\n```"

 analysis_result = structured_llm.invoke(prompt)
 return analysis_result
```

## Deconstructing the Script

This script fortifies our network-dependent analyzer tool against transient failures and inefficient, repeated calls. It introduces three powerful professional patterns.

### Interface vs. Implementation Pattern (Sections 3 & 4)

This is a core software design principle. LangChain's `@tool` decorator needs to inspect a clean function signature to work correctly. However, other decorators like `@retry` can "wrap" the function and hide that signature. We solve this by creating a clean **public interface** (`structured_code_analyzer`) that LangChain can see, which then calls the decorated **internal implementation** (`_analyze_with_retry_and_cache`) where the complex logic lives.

### Automatic Retries with Tenacity (Section 4)

The `@retry` decorator makes our tool antifragile. A transient network glitch that would

normally cause a fatal error will now be handled automatically. The `tenacity` library will catch the exception, wait for an exponentially increasing period, and re-attempt the LLM call. This makes the tool vastly more reliable in a real-world production environment.

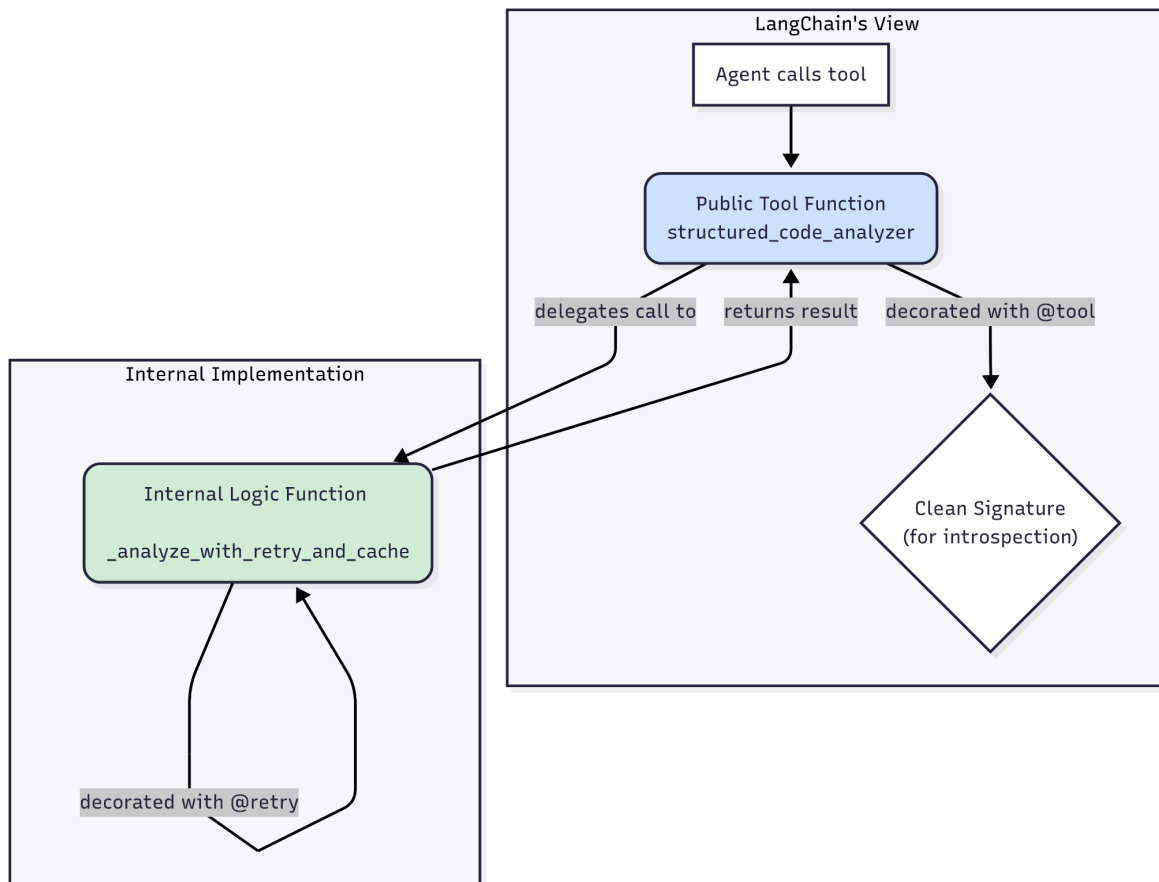


Figure 8-1: The Interface vs. Implementation pattern. The public-facing tool function remains clean for LangChain to inspect, while the complex resilience logic is encapsulated in a decorated internal function.

### Defensive Caching with LRU Cache (Section 4)

The `@lru_cache` decorator is a performance and cost-saving mechanism. Our analyzer is deterministic: the same input code will always produce the same analysis. This decorator

stores the results of recent calls in memory. If the tool is called again with the exact same code snippet, the cached result is returned instantly without making another expensive, time-consuming API call.

#### 4. Create a Test Runner to Demonstrate Resilience

To see our new features in action, we need a special script that can simulate failure conditions and test our caching.

- **File:** 08\_resilient\_agent.py

```
08_resilient_agent.py

import os
import shutil
from dotenv import load_dotenv
from langchain_core.messages import HumanMessage
from langchain_openai import ChatOpenAI

--- 1. Import Components for Testing ---
from agent_graph_factory import build_agent_graph
from tools.retriever_tool import codebase_retriever

We import both the public tool and its "private" internal function
so we can directly test the caching decorator.
from tools.structured_analyzer_tool import (
 structured_code_analyzer,
 _analyze_with_retry_and_cache,
)

load_dotenv()
VECTORSTORE_PATH = "./vectorstore"
VECTORSTORE_BACKUP_PATH = "./vectorstore_backup"

--- 2. Main Demonstration Function ---
def main():
 """Main function to demonstrate the resilient agent's features."""
 print("--- Resilient Agent Demonstration ---")
```

```
--- SCENARIO 1: DEMONSTRATE GRACEFUL FAILURE ---
print("\n--- SCENARIO 1: GRACEFUL FAILURE (VECTORSTORE NOT FOUND)
---")

Arrange: Temporarily hide the vectorstore to simulate a failure
condition.
if os.path.exists(VECTORSTORE_PATH):
 shutil.move(VECTORSTORE_PATH, VECTORSTORE_BACKUP_PATH)
 print(
 f"Temporarily moved vectorstore to
 '{VECTORSTORE_BACKUP_PATH}' to simulate failure."
)

Act: Assemble and run the agent.
tools = [codebase_retriever]
llm = ChatOpenAI(
 base_url="https://openrouter.ai/api/v1",
 api_key=os.getenv("OPENROUTER_API_KEY"),
 model="google/gemini-2.5-flash",
 temperature=0,
)
app = build_agent_graph(llm, tools)
query1 = "What is the purpose of the 'call_model' function?"
initial_state1 = {"messages": [HumanMessage(content=query1)]}
print(f"\nHuman: {query1}")
response1 = app.invoke(initial_state1)

Assert: Observe the agent's helpful error message instead of a
crash.
print(f"Agent: {response1['messages'][-1].content}")

Cleanup: Restore the vectorstore for the next scenario.
if os.path.exists(VECTORSTORE_BACKUP_PATH):
 shutil.move(VECTORSTORE_BACKUP_PATH, VECTORSTORE_PATH)
 print(f"\nRestored vectorstore from backup.")

--- SCENARIO 2: DEMONSTRATE CACHING ---
print("\n\n--- SCENARIO 2: CACHING (DIRECT FUNCTION CALL) ---")
```

```
sample_code = "def my_sample_function(a: int, b: int) -> int:\nreturn a + b"

Act 1: Call the decorated function for the first time.
print(f"\nFirst call to the internal, decorated function...")
response2a = _analyze_with_retry_and_cache(sample_code)
print(f"Result (First Call):\n{response2a}")

Act 2: Call it again with the same input.
print(f"\nSecond call... (Should be cached, no 'Cache miss' message)")
response2b = _analyze_with_retry_and_cache(sample_code)
print(f"Result (Second Call):\n{response2b}")

if __name__ == "__main__":
 main()
```

## Deconstructing the Script

This script is not a standard agent application but a **test runner** designed to prove our new resilience features are working correctly. It simulates real-world failure conditions so we can observe our agent's robust responses.

### Scenario 1: Simulating Graceful Failure

This test validates our refactored `retriever_tool`.

- **Arrange:** We use Python's `shutil` library to temporarily rename the `vectorstore` directory, creating a `FileNotFoundError` condition on purpose.
- **Act:** We run the agent with a normal query.
- **Assert:** We observe the outcome. Instead of a traceback and a crash, the agent's final answer is a helpful message explaining that the vectorstore is missing. This proves that our `try...except` block and lazy loading strategy successfully handled the failure.

### Scenario 2: Demonstrating Caching

This test validates our refactored `structured_analyzer_tool`.

- **Act 1:** We call the internal `_analyze_with_retry_and_cache` function directly. We see the "Cache miss" message printed, indicating that an expensive LLM call was made.
- **Act 2:** We immediately call the *exact same function with the exact same input*.
- **Assert:** This time, the "Cache miss" message does *not* appear. The result is returned instantly. This is definitive proof that the `@lru_cache` decorator intercepted the call and served the result from its memory, saving us time and an API fee.

---

## The Integrated Run & Analysis

Execute our test script to see the resilience patterns in action.

### Run Command:

```
python 08_resilient_agent.py
```

### Expected Console Output:

Pay close attention to the agent's response in Scenario 1 and the "Cache miss" message in Scenario 2.

```
--- Resilient Agent Demonstration ---

--- SCENARIO 1: GRACEFUL FAILURE (VECTORSTORE NOT FOUND) ---
Temporarily moved vectorstore to './vectorstore_backup' to simulate
failure.

Human: What is the purpose of the 'call_model' function?

--- Lazily loading vectorstore... ---
--- Error during vectorstore load or retrieval: [Errno 2] No such file
or directory: 'vectorstore/index.faiss' ---
```

Agent: I encountered an error **while** trying to access the codebase knowledge base. It seems the vectorstore **is not** available. Please ensure the indexing script has been run successfully before I can answer questions about the code.

Restored vectorstore **from** backup.

--- SCENARIO 2: CACHING AND RETRIES (DIRECT FUNCTION CALL) ---

First call to the internal, decorated function...

--- Calling structured\_code\_analyzer (Cache miss **or** new input) ---

Result (First Call):

```
function_name='my_sample_function' description='This function takes two
integers, `a` and `b`, and returns their sum.' args=['a', 'b']
return_type='int'
```

Second call... (Should be cached, no 'Cache miss' message)

Result (Second Call):

```
function_name='my_sample_function' description='This function takes two
integers, `a` and `b`, and returns their sum.' args=['a', 'b']
return_type='int'
```

## Analysis of the Results

The output clearly demonstrates our new resilience features.

- **In Scenario 1**, our simulation worked perfectly. The codebase\_retriever tool caught the FileNotFoundError, and instead of crashing, it returned our custom error string. The main agent then saw this message and synthesized a helpful, diagnostic response for the user. **This is graceful failure in action.**
- **In Scenario 2**, the proof is in the print statements. The line --- Calling structured\_code\_analyzer (Cache miss or new input) --- appeared **only once**. The second time we called the function with the exact same input, the result was returned

---

instantly from the cache, saving us an expensive LLM call and proving our @lru\_cache is working.

---

---

## Summary & Key Takeaways

We have transformed our agent from a capable prototype into a robust system ready for the unpredictability of the real world. This was not just about fixing bugs; it was about architecting for failure as a core design principle.

### Mental Models You've Gained:

- **Resilience as a Feature:** You now see error handling, retries, and caching not as afterthoughts, but as critical, planned features of a production system.
- **Lazy Loading:** You understand the importance of deferring I/O operations (like file access) to avoid brittle, import-time side effects.
- **Interface vs. Implementation:** You have a powerful pattern for separating a tool's clean, public-facing interface from its complex internal logic, especially when using decorators.
- **Idempotency and Caching:** You understand that expensive, deterministic functions (where the same input always produces the same output) are perfect candidates for caching.

### What's Next

Our agent is now strong, smart, and resilient. It has a solid backend and a robust "nervous system." But right now, the only way to interact with it is through a command-line script. To make it truly useful for others, we need to give it a face.

In Lesson 9, we will begin the final phase of our project: building a polished, interactive user interface with Chainlit that allows anyone to use our AI Codebase Analyst.